

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [3]: dataset = pd.read_csv("UberDataset.csv")
dataset.head()
```

```
Out[3]:
```

|   | START_DATE          | END_DATE            | CATEGORY | START          | STOP               | MILES | PURPOSE         |
|---|---------------------|---------------------|----------|----------------|--------------------|-------|-----------------|
| 0 | 01-01-2016<br>21:11 | 01-01-2016<br>21:17 | Business | Fort<br>Pierce | Fort Pierce        | 5.1   | Meal/Entertain  |
| 1 | 01-02-2016<br>01:25 | 01-02-2016<br>01:37 | Business | Fort<br>Pierce | Fort Pierce        | 5.0   | NaN             |
| 2 | 01-02-2016<br>20:25 | 01-02-2016<br>20:38 | Business | Fort<br>Pierce | Fort Pierce        | 4.8   | Errand/Supplies |
| 3 | 01-05-2016<br>17:31 | 01-05-2016<br>17:45 | Business | Fort<br>Pierce | Fort Pierce        | 4.7   | Meeting         |
| 4 | 01-06-2016<br>14:42 | 01-06-2016<br>15:49 | Business | Fort<br>Pierce | West Palm<br>Beach | 63.7  | Customer Visit  |

```
In [4]: dataset.shape
```

```
Out[4]: (1156, 7)
```

```
In [5]: dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1156 entries, 0 to 1155
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   START_DATE  1156 non-null  object
1   END_DATE    1155 non-null  object
2   CATEGORY    1155 non-null  object
3   START       1155 non-null  object
4   STOP        1155 non-null  object
5   MILES       1156 non-null  float64
6   PURPOSE     653 non-null   object
dtypes: float64(1), object(6)
memory usage: 63.3+ KB
```

```
In [6]: dataset.isnull()
```

Out[6]:

|      | START_DATE | END_DATE | CATEGORY | START | STOP  | MILES | PURPOSE |
|------|------------|----------|----------|-------|-------|-------|---------|
| 0    | False      | False    | False    | False | False | False | False   |
| 1    | False      | False    | False    | False | False | False | True    |
| 2    | False      | False    | False    | False | False | False | False   |
| 3    | False      | False    | False    | False | False | False | False   |
| 4    | False      | False    | False    | False | False | False | False   |
| ...  | ...        | ...      | ...      | ...   | ...   | ...   | ...     |
| 1151 | False      | False    | False    | False | False | False | False   |
| 1152 | False      | False    | False    | False | False | False | False   |
| 1153 | False      | False    | False    | False | False | False | False   |
| 1154 | False      | False    | False    | False | False | False | False   |
| 1155 | False      | True     | True     | True  | True  | False | True    |

1156 rows × 7 columns

## Transform Data

In [7]: `dataset['PURPOSE'].fillna("NOT", inplace=True)`

```
In [8]: dataset['START_DATE'] = pd.to_datetime(dataset['START_DATE'],
                                              errors='coerce')
dataset['END_DATE'] = pd.to_datetime(dataset['END_DATE'],
                                      errors='coerce')
```

```
In [9]: from datetime import datetime

dataset['date'] = pd.DatetimeIndex(dataset['START_DATE']).date
dataset['time'] = pd.DatetimeIndex(dataset['START_DATE']).hour

#changing into categories of day and night
dataset['day-night'] = pd.cut(x=dataset['time'],
                             bins = [0,10,15,19,24],
                             labels = ['Morning','Afternoon','Evening','Night'])
```

In [10]: `dataset.dropna(inplace=True)`In [11]: `dataset.drop_duplicates(inplace=True)`

```
In [12]: obj = (dataset.dtypes == 'object')
object_cols = list(obj[obj].index)

unique_values = {}
for col in object_cols:
    unique_values[col] = dataset[col].unique().size
unique_values
```

Out[12]: {'CATEGORY': 2, 'START': 175, 'STOP': 186, 'PURPOSE': 11, 'date': 291}

In [14]: `plt.figure(figsize=(10,5))`

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
sns.countplot(dataset['CATEGORY'])
plt.xticks(rotation=90)

plt.subplot(1,2,2)
sns.countplot(dataset['PURPOSE'])
plt.xticks(rotation=90)
```

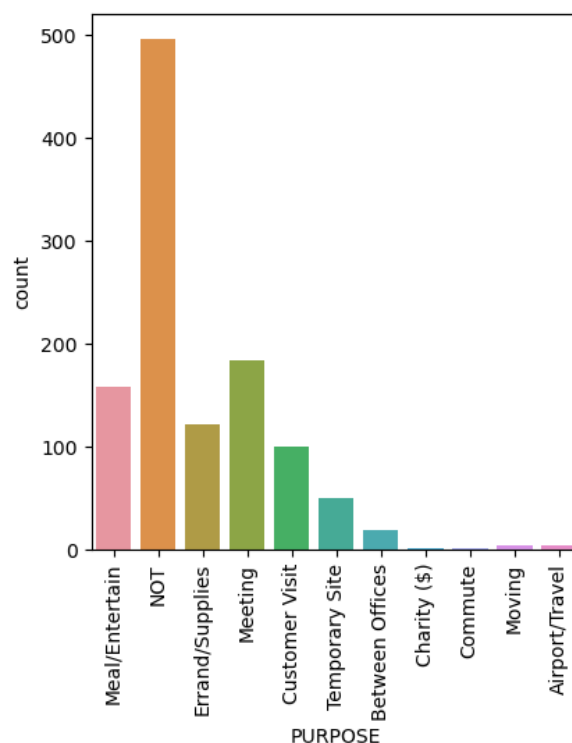
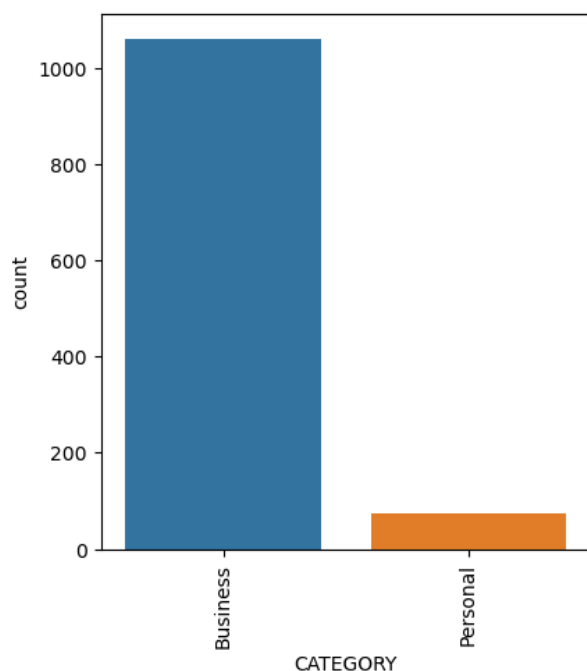
C:\Users\yashc\anaconda3\lib\site-packages\seaborn\\_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

warnings.warn(

C:\Users\yashc\anaconda3\lib\site-packages\seaborn\\_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

warnings.warn(

```
Out[14]: (array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10]),
 [Text(0, 0, 'Meal/Entertain'),
  Text(1, 0, 'NOT'),
  Text(2, 0, 'Errand/Supplies'),
  Text(3, 0, 'Meeting'),
  Text(4, 0, 'Customer Visit'),
  Text(5, 0, 'Temporary Site'),
  Text(6, 0, 'Between Offices'),
  Text(7, 0, 'Charity ($)'),
  Text(8, 0, 'Commute'),
  Text(9, 0, 'Moving'),
  Text(10, 0, 'Airport/Travel')])
```

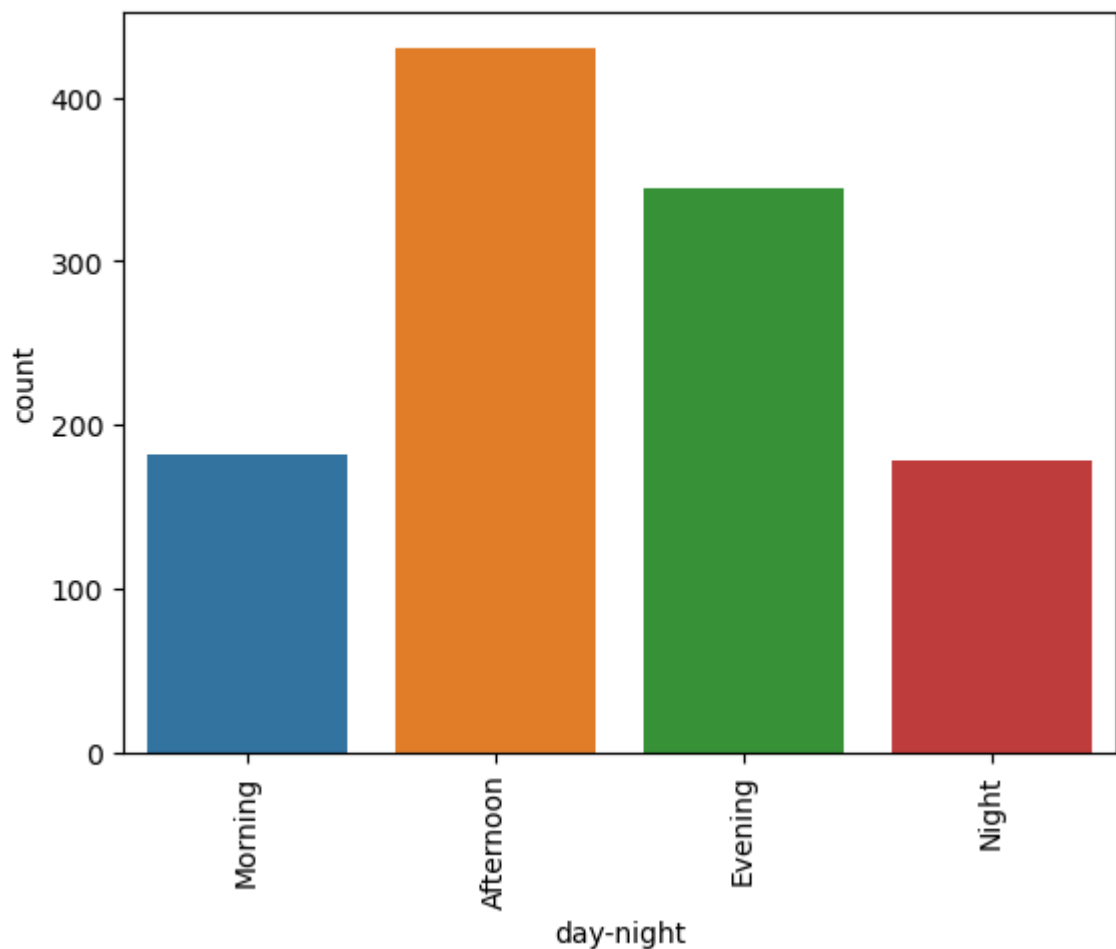


```
In [15]: sns.countplot(dataset['day-night'])
plt.xticks(rotation=90)
```

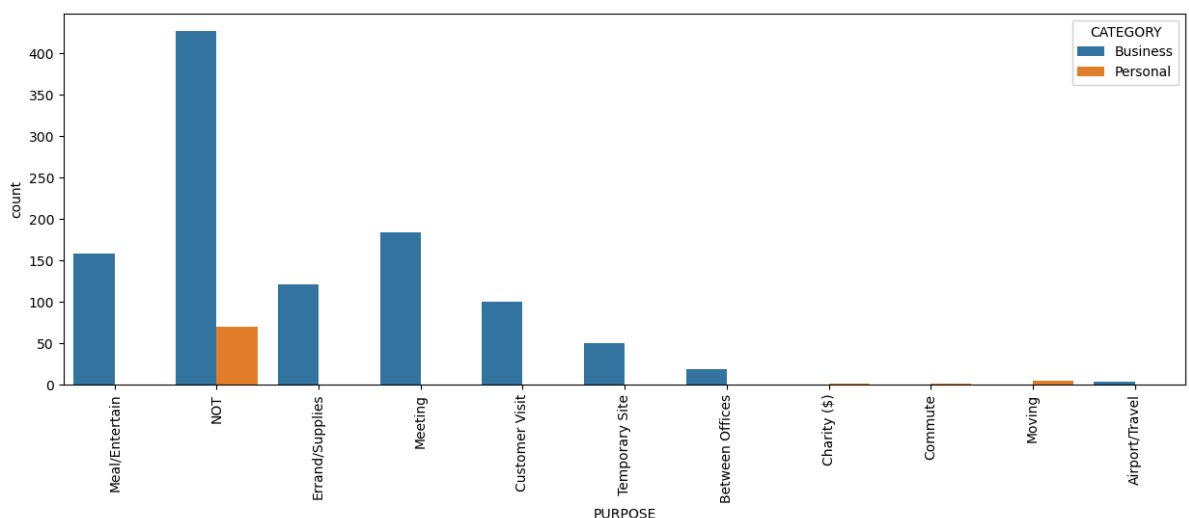
C:\Users\yashc\anaconda3\lib\site-packages\seaborn\\_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

warnings.warn(

```
Out[15]: (array([0, 1, 2, 3]),
 [Text(0, 0, 'Morning'),
  Text(1, 0, 'Afternoon'),
  Text(2, 0, 'Evening'),
  Text(3, 0, 'Night')])
```



```
In [16]: plt.figure(figsize=(15, 5))
sns.countplot(data=dataset, x='PURPOSE', hue='CATEGORY')
plt.xticks(rotation=90)
plt.show()
```



Insights from the above count-plots :

Most of the rides are booked for business purpose.

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js and Meal / Entertain purpose.

Most of the cabs are booked in the time duration of 10am-5pm (Afternoon).

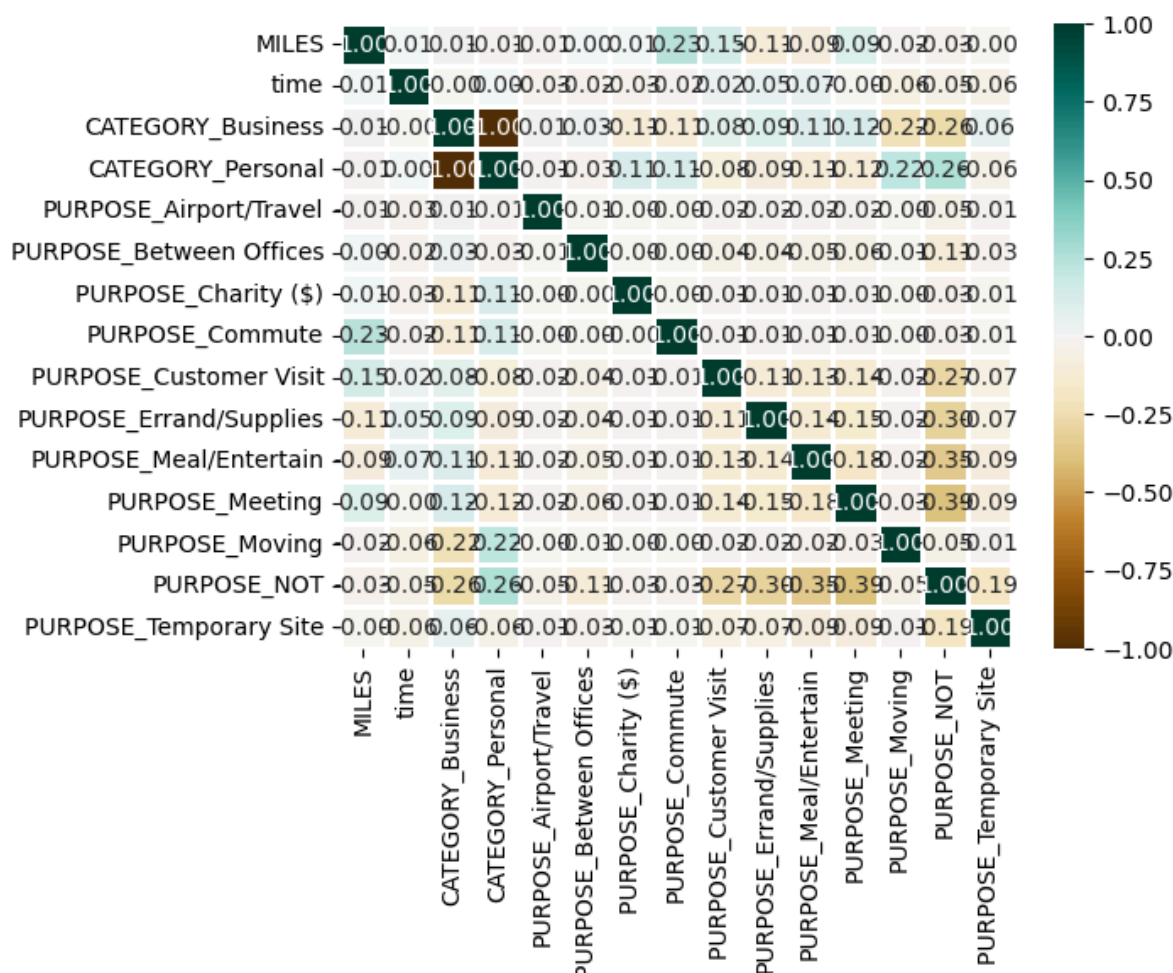
```
In [17]: from sklearn.preprocessing import OneHotEncoder
object_cols = ['CATEGORY', 'PURPOSE']
OH_encoder = OneHotEncoder(sparse=False, handle_unknown='ignore')
OH_cols = pd.DataFrame(OH_encoder.fit_transform(dataset[object_cols]))
OH_cols.index = dataset.index
OH_cols.columns = OH_encoder.get_feature_names_out()
df_final = dataset.drop(object_cols, axis=1)
dataset = pd.concat([df_final, OH_cols], axis=1)
```

```
In [18]: # Select only numerical columns for correlation calculation
numeric_dataset = dataset.select_dtypes(include=['number'])

sns.heatmap(numeric_dataset.corr(),
            cmap='BrBG',
            fmt='.2f',
            linewidths=2,
            annot=True)

# This code is modified by Susobhan Akhuli
```

Out[18]: <AxesSubplot:>



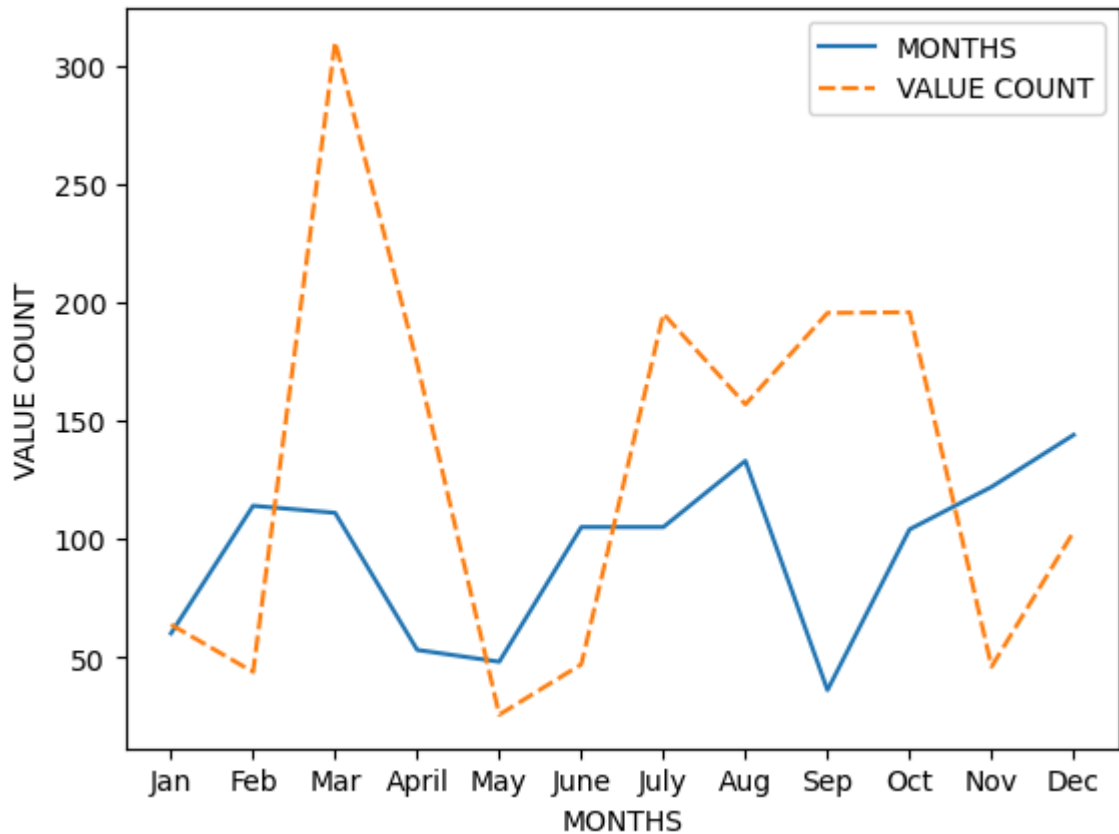
```
In [19]: dataset['MONTH'] = pd.DatetimeIndex(dataset['START_DATE']).month
month_label = {1.0: 'Jan', 2.0: 'Feb', 3.0: 'Mar', 4.0: 'April',
               5.0: 'May', 6.0: 'June', 7.0: 'July', 8.0: 'Aug',
               9.0: 'Sep', 10.0: 'Oct', 11.0: 'Nov', 12.0: 'Dec'}
dataset["MONTH"] = dataset.MONTH.map(month_label)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js  
 month = dataset.MONTH.map(month\_label, na\_action='ignore', sort=False)

```
# Month total rides count vs Month ride max count
df = pd.DataFrame({"MONTHS": mon.values,
                  "VALUE COUNT": dataset.groupby('MONTH',
                                                  sort=False)['MILES'].max()})

p = sns.lineplot(data=df)
p.set(xlabel="MONTHS", ylabel="VALUE COUNT")
```

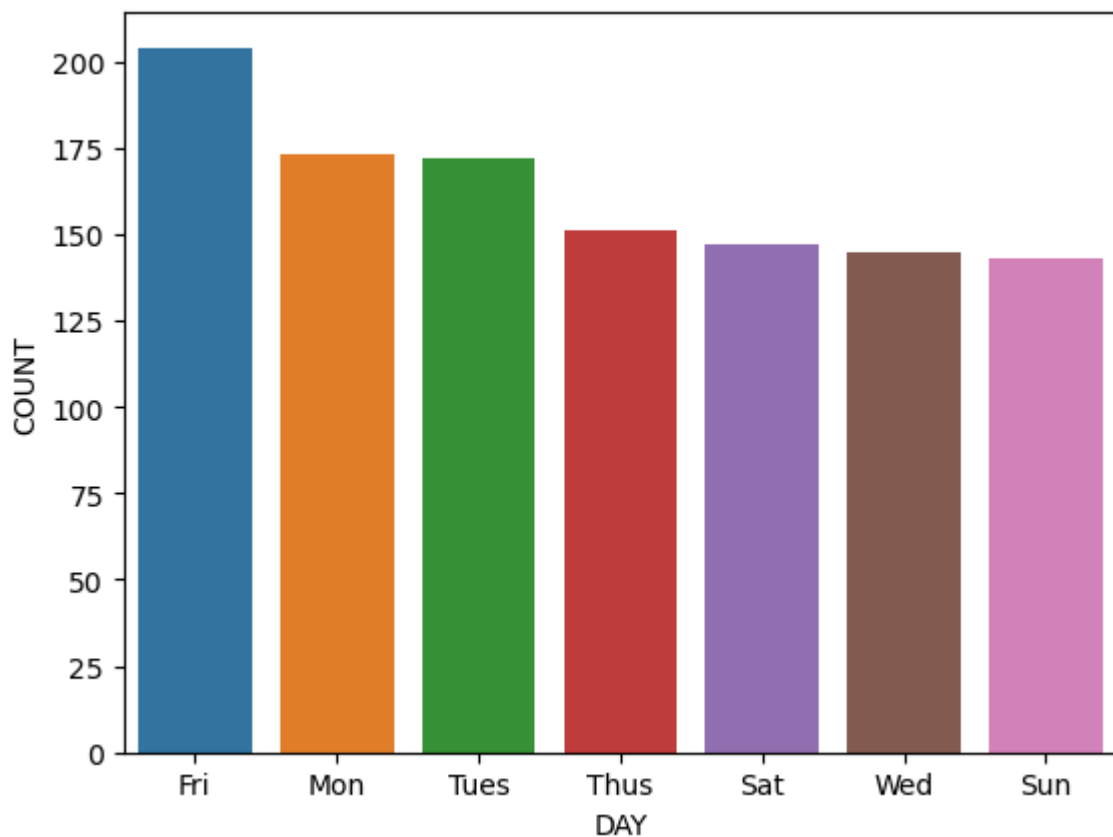
Out[19]: [Text(0.5, 0, 'MONTHS'), Text(0, 0.5, 'VALUE COUNT')]



```
In [20]: dataset['DAY'] = dataset.START_DATE.dt.weekday
day_label = {
    0: 'Mon', 1: 'Tues', 2: 'Wed', 3: 'Thus', 4: 'Fri', 5: 'Sat', 6: 'Sun'
}
dataset['DAY'] = dataset['DAY'].map(day_label)
```

```
In [21]: day_label = dataset.DAY.value_counts()
sns.barplot(x=day_label.index, y=day_label);
plt.xlabel('DAY')
plt.ylabel('COUNT')
```

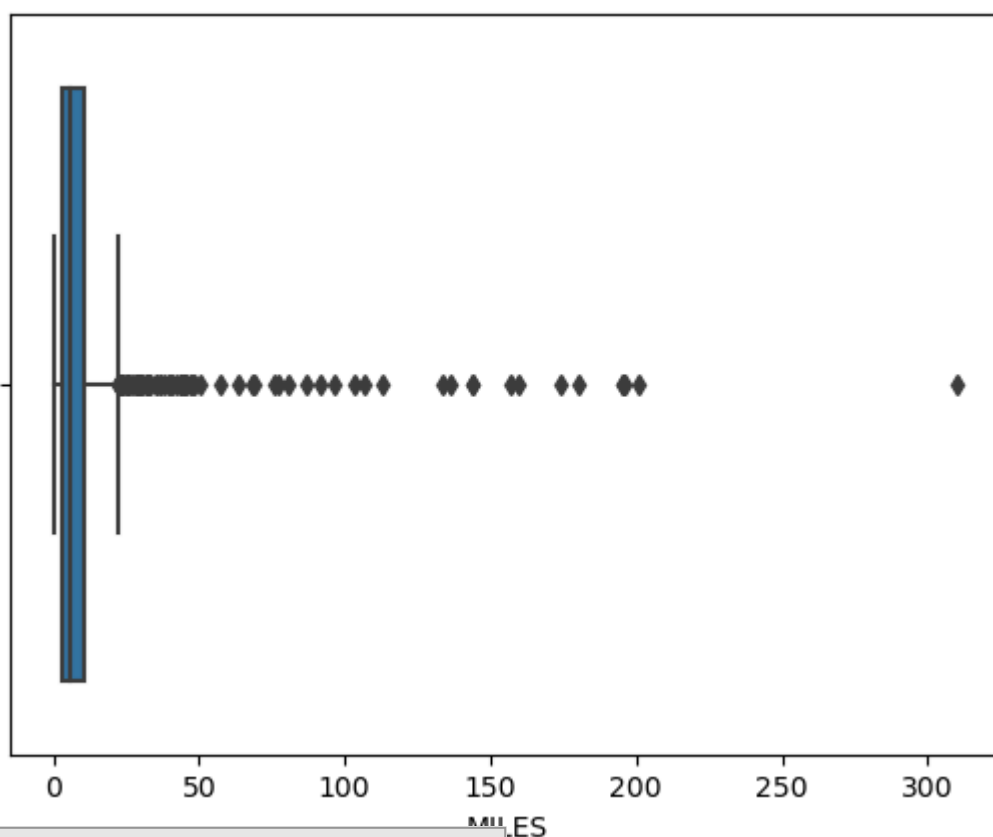
Out[21]: Text(0, 0.5, 'COUNT')



```
In [22]: sns.boxplot(dataset['MILES'])
```

C:\Users\yashc\anaconda3\lib\site-packages\seaborn\\_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

```
Out[22]: <AxesSubplot:xlabel='MILES'>
```

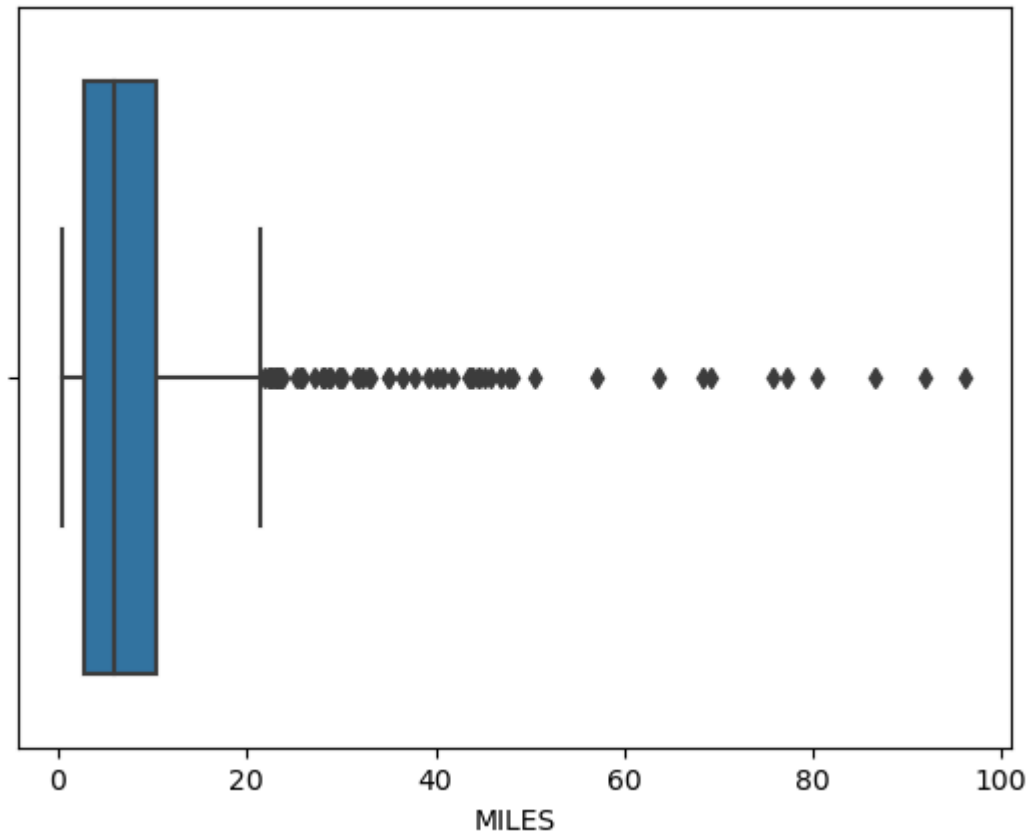


Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

In [23]: `sns.boxplot(dataset[dataset['MILES']<100]['MILES'])`

C:\Users\yashc\anaconda3\lib\site-packages\seaborn\\_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

Out[23]: `<AxesSubplot:xlabel='MILES'>`

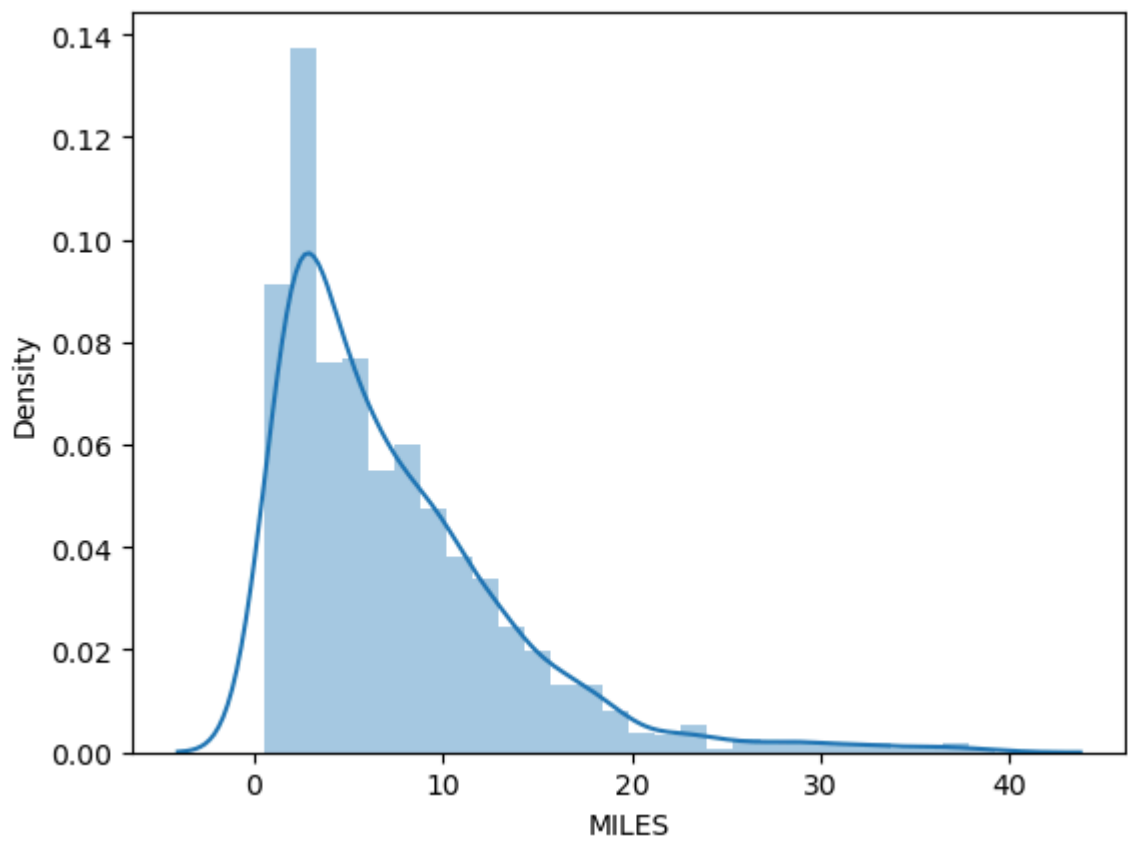


In [24]: `sns.distplot(dataset[dataset['MILES']<40]['MILES'])`

C:\Users\yashc\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

Out[24]: `<AxesSubplot:xlabel='MILES', ylabel='Density'>`





In [ ]: